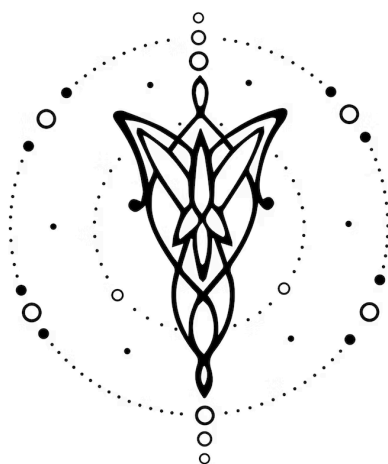




Especificación – Synchro

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

1. Equipo	1
2. Repositorio	1
3. Dominio	1
4. Construcciones	2
5. Casos de Prueba	2
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Manuel	Ahumada	64.677	maahumada@itba.edu.ar
Josefina	Gonzalez Cornet	64.550	jgonzalezcornet@itba.edu.ar
Juan Ignacio	Lategana	64.476	jategana@itba.edu.ar
Nicolás	Priotto	64.663	npriotto@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [Synchro](#).

3. Dominio

Desarrollar un lenguaje que permita especificar y simular ejercicios de sincronización con threads utilizando semáforos. El lenguaje debe permitir la declaración de semáforos y variables compartidas, la definición de threads, imprimir a salida estándar, generar tiempos de espera y la utilización de primitivas de sincronización como down y up.

El objetivo de este lenguaje es facilitar la prueba y validación de soluciones a problemas clásicos de concurrencia, como lectores-escritores, la exclusión mutua y la sincronización de threads. El grupo de usuarios esperado está compuesto por personas con conocimientos previos sobre sincronización, que comprenden los conceptos de concurrencia y paralelismo, así como el funcionamiento de los semáforos en C. El código escrito en este DSL se traducirá a C, donde será compilado y ejecutado para evaluar su correcto funcionamiento.

4. Construcciones

El lenguaje desarrollado ofrecerá las siguientes construcciones, prestaciones y funcionalidades:

- (I). Declaración de semáforos con valores iniciales.
- (II). Declaración de variables compartidas de tipo int, boolean, y string.
- (III). Definición de threads con bloque de instrucciones, a partir de una función thread().
- (IV). Definición de funciones, a las que se les puede pasar parámetros.
- (V). Uso de primitivas de sincronización: down(sem) y up(sem).
- (VI). Operaciones aritméticas básicas sobre variables compartidas.
- (VII). Estructuras de control como if, while y for.
- (VIII). Imprimir a salida estándar variables enteras, booleanas, y strings, mediante una función print().
- (IX). Imprimir a salida estándar el valor de un semáforo.
- (X). Traducción del código del DSL a C.
- (XI). Posibilidad de realizar comentarios de línea en el código.
- (XII). Posibilidad de realizar un sleep(), para ver mejor el funcionamiento de la sincronización.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que implemente el problema de lectores-escritores.
- (II). Un programa que implemente la exclusión mutua entre dos threads.
- (III). Un programa que sincronice tres threads en un orden específico.
- (IV). Un programa donde varios threads incrementan una variable compartida sin exclusión mutua.
- (V). Un programa donde se asegura acceso controlado a una región crítica.
- (VI). Un programa que implemente el problema de filósofos comensales sin inanición.
- (VII). Un programa que implemente el problema de filósofos comensales con inanición.
- (VIII). Un programa que genere un deadlock con un semáforo entre 2 threads.
- (IX). Un programa que haga down a un semáforo indefinidamente.
- (X). Un programa donde dos threads alternan acceso a una variable compartida en orden estricto.
- (XI). Un programa donde un escritor espera a que todos los lectores terminen antes de escribir.
- (XII). Evaluación de expresiones aritméticas (+, -, *, /, ++).
- (XIII). Evaluación de las estructuras de control (if, while, for).
- (XIV). Asignación y uso de variables booleanas y strings.
- (XV). Verificación de salida estándar para cada tipo de dato.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado.
- (II). Un programa que intente hacer down sobre un semáforo no declarado.
- (III). Un programa que intente modificar una variable compartida sin declararla.
- (IV). Un programa con un if sin bloque de instrucciones válido.
- (V). Un programa que defina dos semáforos con el mismo nombre.
- (VI). Un programa que use operadores aritméticos incorrectamente en una condición de if.
- (VII). Un programa que use palabras reservadas como nombres de variables o threads.
- (VIII). Un programa que no tenga la función “main”.

6. Ejemplos

Ejemplo 1: Implementación del problema de lectores-escritores

```
sem mutex = 1
sem queue = 1
sem db = 1
int rc = 0
int wc = 0

reader() {
    while(1) {
        down(queue)
        down(mutex)
        rc++
        if (rc == 1) {
            down(db)
        }
        up(queue)
        up(mutex)

        print("LEYENDO BASE DE DATOS")

        down(mutex)
        rc--
        if(rc == 0) {
            up(db)
        }
        up(mutex)
        print("USANDO BASE DE DATOS")
    }
}
```

```

    }
}

writer() {
    while(1) {
        down(queue)
        down(db)
        up(queue)
        print("ESCRIBIENDO EN LA BASE DE DATOS")
        sleep(5) // Tiempo de "escritura" en la base de datos
        up(db)
    }
}

main {
    for(5) thread(writer())
    for(1) thread(reader())
}

```

Ejemplo 2: Exclusión mutua entre dos threads

```

sem mutex = 1

A() {
    while(1) {
        down(mutex)
        print("ACCESO SECCIÓN CRÍTICA - A")
        up(mutex)
    }
}

B() {
    while(1) {
        down(mutex)
        print("ACCESO SECCIÓN CRÍTICA - B")
        up(mutex)
    }
}

main {
    thread(A())
    thread(B())
}

```

Este lenguaje permitirá evaluar y testear soluciones a problemas de concurrencia de forma sencilla y estructurada.